LLM 경량화와 소형 언어 모델

더 작은 메모리 · 더 빠른 추론 · 실제 배포를 위한 설계



KUGGLE 14기 이재헌

LLM의 발전

모델의 성능 향상과 함께 학습·활용·특화 방식도 다양해졌다

① 학습과 정렬

사전학습: 대규모 텍스트에서 패턴 학습

SFT: 지시·답변 데이터로 작업 적응

RLHF/DPO: 선호에 맞게 응답 조정

② 기능의 확장

RAG: 외부 문서 검색 후 답변

에이전트: 도구 호출과 작업 수행

멀티모달: 텍스트 외 입력까지 처리

③ 효율적인 특화

PEFT: 전체가 아닌 일부만 학습

LoRA: 저랭크 행렬로 변화량 학습

기존 가중치를 대부분 고정해 절약

핵심 질문 | 더 강력해진 모델을 어떻게 더 적은 자원으로 사용할까?

규모의 세 가지 부담

모델 크기 증가가 학습뿐 아니라 추론과 배포 비용까지 높인다

컴퓨팅·운영 비용

GPU 메모리와 연산량 요구 증가
서비스 동시 요청 수에 따라 비용 증가
지연시간과 전력 소비도 고려

미세조정의 위험

작은 특화 데이터에 과적합 가능
기존 지식이 손상되는 파국적 망각
전체 재학습은 메모리·시간 부담

저장·배포 비용

70B × 2 byte = 약 140GB
이는 FP16 가중치만의 이론적 크기
실행 시 KV 캐시 등 추가 메모리 필요

예시: 70B 가중치 | FP16 약 140GB → INT4 이론상 약 35GB (+ 부가 정보)

LLM의 효율성: 두 가지 접근

이미 만든 모델을 압축하는 경량화 vs 작은 모델을 효율적으로 설계하는 SLM

경량화 · Compression

출발점: 이미 학습된 큰 모델

양자화: 가중치 표현 비트 축소

가지치기: 불필요한 연결·구조 제거

지식 증류: 작은 모델에 행동 전달

SLM · Small Language Model

출발점: 작은 모델을 목적에 맞게 설계

양질의 데이터와 학습 목표 선택

특정 업무·기기 환경에 최적화

작은 모델에 추가 경량화 적용 가능

두 접근은 경쟁 관계가 아니라 함께 적용할 수 있는 전략

경량화와 SLM은 어떻게 연결될까?

원본 모델의 크기를 줄이는 과정과 작은 모델의 설계 과정을 구분한다

대형 모델

양자화

가지치기

→

경량화 모델

메모리 절감

성능 검증

→

실제 배포

로컬 실행

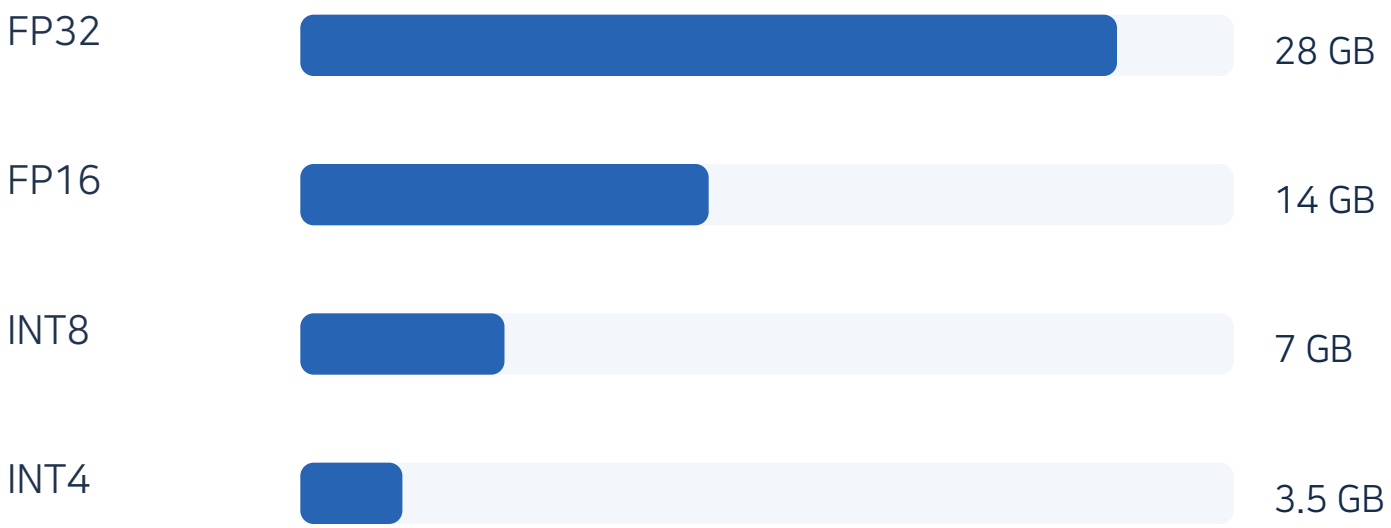
비용 절감

별도 경로: 고품질 데이터 → 작은 모델 학습(SLM) → 추가 양자화

모델이 작아졌다는 사실만으로 속도·정확도가 보장되지는 않는다.

양자화의 원리

파라미터 개수는 유지하고 숫자를 저장하는 비트 수를 줄인다



7B 모델 가중치 예시

FP32 28GB

FP16 14GB

INT8 7GB

INT4 3.5GB

계산: 7 × 10 파라미터 × 비트 수 ÷ 8 | 부가 메타데이터·KV 캐시는 제외

PTQ와 QAT: 언제 양자화할까?

압축을 학습 후 적용하는지, 학습 과정에 반영하는지의 차이

PTQ · Post-Training Quantization

① 일반 정밀도로 모델 학습 완료
② Calibration으로 양자화 범위 추정
③ 저정밀 가중치로 변환 후 평가
장점: 빠른 적용 / 단점: 성능 저하 가능

QAT · Quantization-Aware Training

① 학습 중 가짜 양자화 연산 적용
② 반올림·클리핑 오차에 적응
③ 학습 후 저정밀 모델로 배포
장점: 정확도 유지 / 단점: 추가 학습 비용

선택 기준: 시간·데이터·훈련 자원이 부족하면 PTQ부터 검토

고급 양자화 기법

모든 4비트 양자화가 같은 방식으로 정보를 보존하는 것은 아니다

NF4

정규분포형 가중치를 위한 4비트 형식

동일한 16개 값으로 균등 분할하지 않음

QLoRA에서 자주 활용되는 데이터 형식

이중 양자화 · Calibration

이중 양자화: 스케일 정보도 압축

Calibration: 대표 입력으로 범위 추정

기법마다 적용 단계와 목적이 다름

GPTQ

학습 완료 모델에 적용하는 PTQ

출력 오차를 줄이도록 가중치 보정

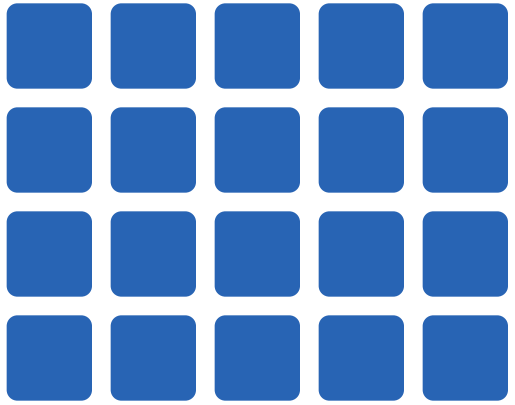
모델·하드웨어별 지원 여부 확인

주의: NF4·GPTQ·Calibration은 PTQ와 QAT의 동일한 하위 분류가 아니다.

가지치기: 어떤 가중치를 없앨까?

0으로 만든 가중치가 많아도 실제 추론이 빨라지는 것은 아니다

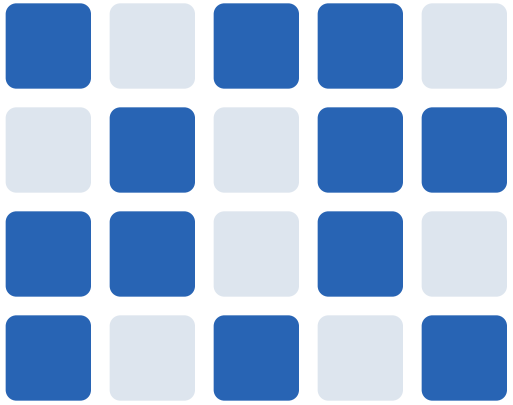
원본 행렬



모든 연결 유지

연산량이 큼

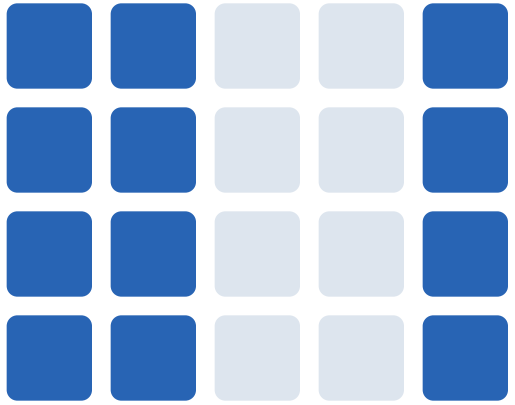
비구조적



불규칙한 0

희소 커널 필요

구조적



열·헤드 단위 제거

밀집 연산에 유리

Wanda: 가중치만 보지 않는 가지치기

가중치 크기와 입력 활성값을 함께 고려해 제거 우선순위를 정한다

중요도 점수의 직관

Wanda 점수 |가중치| × 입력 활성 크기

가중치가 작아도 자주 큰 입력을 받으면

출력에 미치는 영향이 클 수 있음

출력 채널별로 낮은 점수부터 제거

간단한 계산 예시

연결 A: |0.2| × 10 = 2.0

연결 B: |0.8| × 1 = 0.8

→ B를 먼저 제거하는 판단 가능

※ 예시 값, 실제 점수는 활성 통계 사용

지식 증류: 큰 모델이 작은 모델을 가르친다

Teacher의 정답뿐 아니라 후보별 확률 분포도 학습 신호가 된다

Teacher · 큰 모델

입력: “동물 사진 분류”

고양이 80% / 강아지 15%

햄스터 5%

후보 간 유사성 정보를 제공

Soft targets + 온도 T

Student · 작은 모델

Teacher의 분포를 모방

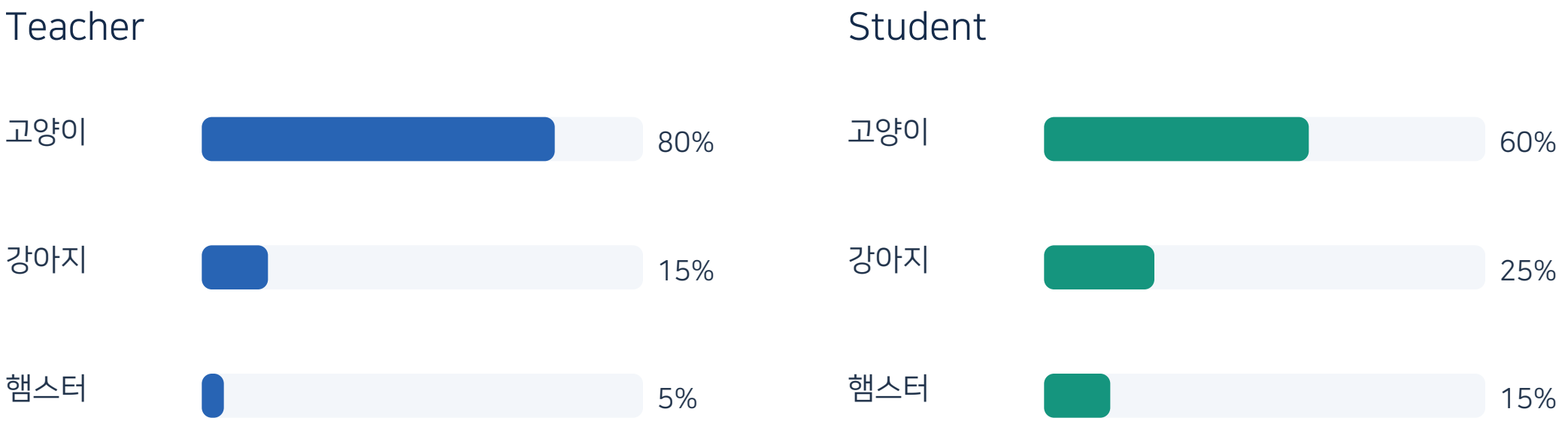
정답 레이블과 함께 학습 가능

작은 크기에서 성능 유지 시도

정답 1개보다 풍부한 정보: “고양이와 강아지는 더 비슷하다”

KL Divergence와 증류 최적화

Teacher와 Student가 출력하는 확률 분포를 얼마나 다르게 보는가?



기본 증류 손실: KL(Teacher || Student) | T^2 배율과 정답 손실을 함께 사용 가능

세 가지 경량화 기법의 통합

압축 대상이 서로 다르므로 목적에 맞춰 조합한다

양자화 · 표현 축소

32/16비트 → 8/4비트

메모리·저장 용량 절약

파라미터 수 자체는 동일

가지치기 · 구조 축소

연결·헤드·채널 일부 제거

희소 연산 지원 시 속도 이점

정확도·하드웨어 영향 검증

증류 · 지식 이전

큰 Teacher → 작은 Student

작은 모델을 별도로 학습

학습 비용은 추가로 발생

예: Teacher → Student 증류 → INT4 양자화 → 실제 장치에서 평가

SLM: 작은 모델을 잘 설계하는 전략

단순 축소가 아니라 데이터·학습 목표·활용 범위를 선택하는 문제

작업 범위 정의

고객 상담 / 문서 분류 / 코드 보조
모든 분야를 잘할 필요는 없음
목표 도메인에 용량 집중

학습 데이터 품질

중복·오류·노이즈 정리
고품질 예제와 설명 중심 학습
평가 데이터 누출 방지

배포 조건 최적화

메모리·전력·지연시간 한도
온디바이스 실행 가능성
업무 정확도와 안전성 검증

작은 모델 = 항상 저성능이 아님 | 범용성과 특화 성능은 구분

LLM vs SLM: 어떤 모델을 선택할까?

최대 성능만이 아니라 비용·속도·개인정보·운영 환경을 함께 비교

비교 기준	대형 LLM	소형 SLM
범용 지식	넓은 범위에 유리	업무 특화에 유리
메모리·비용	상대적으로 큼	상대적으로 작음
지연시간	인프라에 따라 다름	로컬에서 낮출 여지
배포 방식	클라우드 중심 가능	온디바이스 적합 가능
주요 위험	비용·외부 의존	복잡한 작업 한계

로컬 AI: 모델을 내 컴퓨터에서 실행하기

파일 형식·추론 엔진·모델 관리 도구는 서로 다른 역할을 맡는다

Hugging Face

모델·데이터셋 탐색

라이선스·모델 카드 확인

→

GGUF / llama.cpp

GGUF: 배포 파일 형식

llama.cpp: 로컬 추론 엔진

→

Ollama

모델 다운로드·실행 관리

로컬 API로 앱과 연결

실행 가능 여부는 RAM/VRAM, 양자화 수준, 문맥 길이, CPU/GPU에 좌우

경량화 모델의 평가: 성능과 효율성

압축 전후 품질 변화와 실제 자원 사용량을 함께 측정한다

표준 벤치마크

MMLU: 지식·문제 해결

HumanEval: 코드 생성 테스트

MT-Bench: 다회 대화 평가

벤치마크마다 측정 능력이 다름

시스템 효율성

가중치 크기와 실제 메모리

첫 토큰 지연(TTFT)

초당 생성 토큰 수(TPS)

전력·동시 처리량

공정한 비교

같은 데이터·프롬프트 사용

동일 장비·배치·문맥 길이

품질 손실 대비 절약량 확인

수치 하나로 결론 내리지 않기

애플리케이션 특화 평가

일반 시험 점수보다 실제 업무에서 올바르게 답하는지가 중요하다

예: 사내 문서 기반 Q&A

질문: “출장비 상한은 얼마인가?”
검색된 사내 규정 문서에 근거해야 함
근거 없는 수치 생성은 실패
업무 정확도·보안·응답시간 측정

RAG 평가 지표

Faithfulness: 답변이 근거에 충실한가?
Answer Relevancy: 질문에 답하는가?
검색 Recall: 필요한 문서를 찾았는가?
Ragas·DeepEval 등으로 점검 가능

배포 전에는 실제 사용자 질문을 반영한 자체 테스트셋이 필요하다.

효율성 혁명: 전체 흐름 정리

비용 문제에서 출발해 압축·작은 모델·로컬 실행·검증으로 연결된다

문제	기법	대안	배포	검증
비용·메모리	양자화·가지치기·증류	SLM 설계	로컬·클라우드	품질·속도·비용

성공 기준: 작은 모델이 아니라, 필요한 성능을 더 적은 비용으로 제공하는 모델

AI의 미래: 가장 큰 모델보다 적합한 모델

용도와 제약 조건에 맞춰 LLM·SLM·경량화 모델을 조합한다

클라우드 LLM이 적합할 때

광범위한 지식과 복잡한 작업
유연한 모델 업그레이드가 중요
서버 기반 운영이 허용되는 환경

로컬 SLM이 적합할 때

오프라인·저지연 처리가 중요
민감 정보의 외부 전송 최소화
정해진 업무를 반복 수행하는 환경

최종 선택 = 정확도 × 운영 안정성 × 자원 효율 × 개인정보 요구